Tenant Isolation

silo, pool or bridge: chosen layer by layer

Watch the 4-min video

In one line: tenant isolation is a dial, not a switch: **silo** gives each tenant dedicated resources, **pool** shares everything and isolates in code, and real SaaS products set the dial per layer (compute, data, network) and per tier, with a routing layer that hides the difference from the product.

1 The bank's three questions

Your SaaS serves 3,000 small businesses on shared infrastructure. A bank wants to sign up, and its security questionnaire asks three things: **where exactly is our data** (which database, region and backups), **who shares our servers**, and **what happens if another customer's workload goes wild** or a bug crosses tenant boundaries. Each answer is set by your isolation model, and that model also sets your cost per tenant, how you deploy, and how fast you can onboard a new customer.

Two different risks hide in those questions. **Data isolation**: can tenant A ever read or change tenant B's data? **Performance isolation**: can tenant A's load slow down tenant B (the noisy neighbour, lesson 44)? A design can be strong on one and weak on the other. Note also that isolation is not the same as authentication: a correctly logged-in user is still the main threat if the code forgets to check whose data they are asking for (lesson 43).

2 Silo, pool and bridge

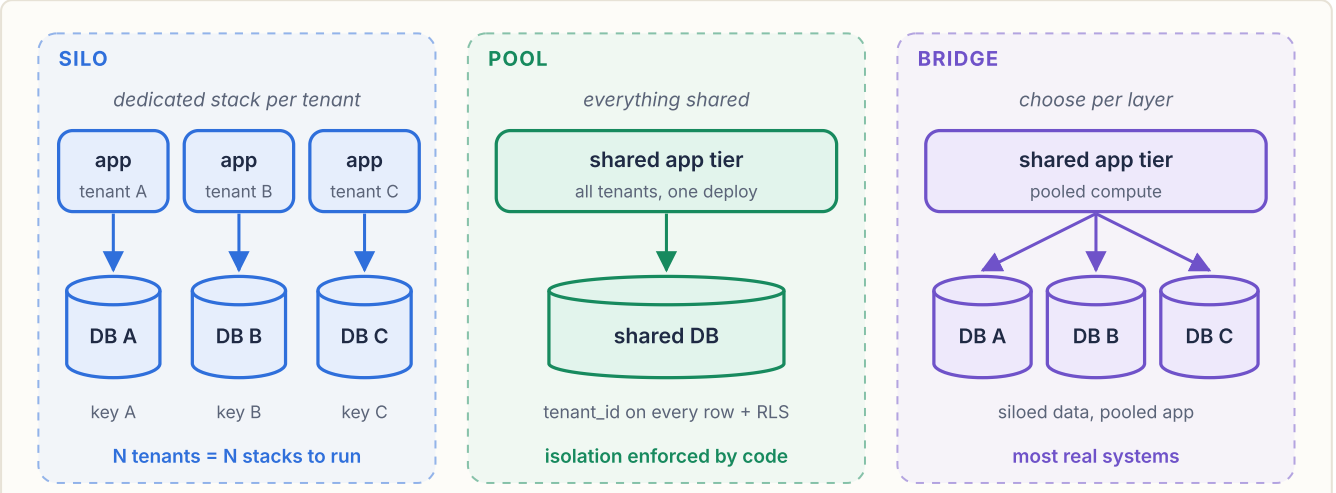


Figure 1. The three models from the AWS SaaS Lens. The bridge mixes them layer by layer; here the app is pooled and the data siloed.

Silo: each tenant gets its own stack: compute, database, often its own cloud account or subscription and network. A bug in one tenant's stack cannot touch another's data, blast radius is one tenant, and cost per tenant is easy to measure. But you pay for idle capacity in every stack, onboarding means provisioning infrastructure (minutes to hours, so it must be automated with infrastructure as code), and every release must roll out to N stacks. Silo only works if every stack runs the **same code version**; per-tenant forks turn a SaaS into a consulting business.

Pool: all tenants share the same services and databases, and every row and request carries a `tenant_id`. One deployment, high utilisation (the peaks of different tenants rarely line up), onboarding is an insert into a table. But isolation is **logical**: one missing filter leaks data, one heavy tenant slows everyone, and per-tenant cost must be estimated from metering (lesson 45) rather than read off a bill.

Bridge: choose silo or pool separately for each layer. Common combinations: pooled web and API tier with a database (or schema) per tenant; pooled compute with siloed storage for regulated tenants; pooled everything except a dedicated worker pool for heavy batch tenants.

	Silo	Pool	Bridge (app pooled, data siloed)
Data isolation	physical, easy to prove	logical: code + RLS	physical for data
Noisy neighbour	none (own resources)	main risk	in the shared app tier
Cost per tenant	high, includes idle capacity	lowest	medium
Onboarding	provision a stack	insert a row	create a database
Deploy / migrate	N times	once	app once, schema N times
Cost attribution	read off the cloud bill	needs metering	partly direct
Fits	regulated, very large tenants	many small tenants	most growing SaaS

3 Isolation happens at three levels

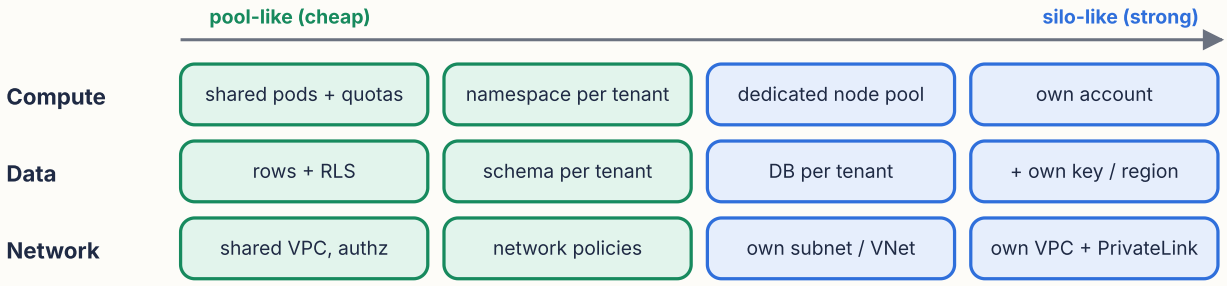


Figure 2. Each layer has its own ladder of options. Pick a rung per layer, and per tier, instead of one model for the whole system.

- **Compute.** Shared pods with per-tenant quotas and concurrency limits; a Kubernetes namespace per tenant with a `ResourceQuota` and `NetworkPolicy`; a dedicated node pool (taints and tolerations keep other tenants' pods off it, lesson 57); or a separate cloud account or subscription, the strongest boundary because even the cloud IAM is separate. Remember that containers share the host kernel (lesson 55): for untrusted tenant code, use VM-level isolation such as gVisor, Kata or Firecracker micro-VMs.
- **Data.** Rows with `tenant_id` plus row-level security, a schema per tenant, or a database per tenant (lesson 42). Add **per-tenant encryption keys** (envelope encryption with a key in a KMS): the tenant's data keys are wrapped by its own master key, the key can be customer-managed, and deleting it on offboarding makes every copy, including backups, unreadable ("crypto-shredding").
- **Network.** In a pool, tenants never get network access to your internals, so network isolation is about your own components: network policies so a compromised service can't reach every database. In a silo, a VPC or VNet per tenant, and private connectivity (AWS PrivateLink, Azure Private Link) when the tenant connects from its own network.

Do not forget the layers in between: caches (key prefix per tenant), object storage (prefix or bucket per tenant, with scoped credentials), search indexes, queues, logs and analytics. A tenant's data leaks

through whichever layer you forgot.

4 Tier it, and route by tenant

The practical answer is **tiering**: free and SMB tenants go into the pool, where cost matters most; enterprise and regulated tenants get silos, and their higher price pays for the dedicated resources and the residency (lesson 46) and compliance story. A **tenant router** resolves the tenant from the signed token or the subdomain, looks up its tier and location in a **tenant catalog**, and forwards the request. The catalog is read on every request, so it is cached at the router; it is also the place you change when a tenant moves between tiers.

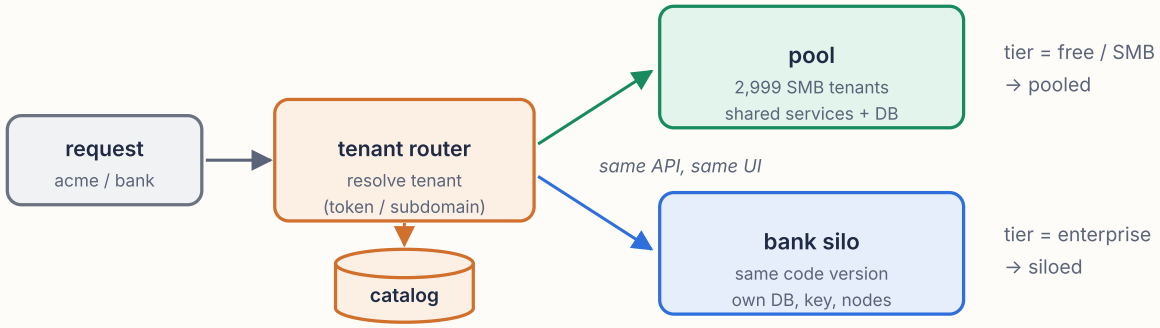


Figure 3. A router plus a catalog make tiers invisible to the product: the same code, API and UI everywhere, different isolation underneath.

Because tenants upgrade, design the **move** from pool to silo from day one: keep `tenant_id` on every row even in siloed databases, so the same code and the same migration tooling (lesson 42) work in both places. At the extreme, the silo becomes a full deployment stamp (lesson 50), and the router becomes a global routing layer.

WORKED EXAMPLE: ENTERPRISE DEDICATED, SMB POOLED

3,000 SMB tenants share the pooled services and one Postgres cluster: every table has `tenant_id`, every query is filtered by the repository layer, and row-level security is the backstop. The bank runs the **same container images and release**, but on its own dedicated node pool (tainted so no other pods land there), with its own database and its own customer-managed KMS key. The tenant router reads `tenant=bank → tier=enterprise, db=pg-bank-01, pool=np-bank` from the catalog; every other tenant maps to the pool. The bank's answers: its data lives in one named database, in one region, encrypted with a key it controls; nobody else's workload runs on its nodes; and a runaway SMB tenant can at worst slow the shared API tier, which has per-tenant rate limits.

TRADE-OFFS

- Silo buys isolation and simple compliance with cost and operational effort that grow with every tenant.
- Pool buys efficiency and one-step deploys with isolation that depends on code being right every time.
- Bridge and tiering get both, but you now run two shapes of system and need routing, a catalog and migration tooling.
- The stronger the isolation, the slower onboarding is, unless provisioning is fully automated.

COMMON MISTAKES

- Choosing one model for the whole system instead of per layer and per tier.
- Letting siloed tenants drift onto custom code versions or hand-made infrastructure.
- Isolating the database but sharing an unprefixed cache, bucket or search index.
- Selling "dedicated" to an enterprise without a tested way to move it there from the pool.

5 Key terms

Silo model: dedicated resources per tenant

Bridge model: silo or pool chosen per layer

Crypto-shredding: deleting a tenant's key to make its data unreadable

Pool model: shared resources; tenant_id isolates

Tenant catalog: maps each tenant to its tier and location

Blast radius: how many tenants one failure affects

6 Interview questions

Q1. Compare silo and pool tenancy models. BASIC

Silo gives each tenant dedicated resources: strong isolation and easy compliance, but higher cost and heavier operations. Pool shares everything: cheap and simple, but isolation relies on code and noisy neighbours are a risk.

Q2. What do most systems do? BASIC

A bridge, or tiering: pool smaller tenants, silo enterprise ones, and route each tenant to its tier through a catalog, with the same code everywhere.

Q3. At which levels can you isolate tenants? BASIC

Compute (quotas, namespaces, node pools, accounts), data (rows + RLS, schema, database, per-tenant keys) and network (policies, separate VPCs, private links).

Q4. A bank asks for its data to be isolated, but you run a pooled system. What do you offer? SCENARIO

A bridge: same application code, but a dedicated database with its own customer-managed key and optionally a dedicated node pool, routed by the tenant catalog and priced as an enterprise tier.

Q5. Customers on the free tier keep growing. When does a tenant leave the pool? SCENARIO

When its load or data share starts to hurt others or it needs residency or compliance guarantees; move it with a bulk copy plus change data capture and a catalog flip (lesson 42).

Q6. Is a separate database per tenant enough isolation? TRICKY

Not alone: caches, object storage, search indexes, queues, logs and shared compute can still mix tenants, and app code still decides which database to open.

Q7. Why is per-tenant cost harder in the pool model? TRICKY

Shared resources produce one bill, so you must attribute cost from per-tenant metering (requests, CPU time, storage), not read it from the cloud invoice.

Q8. Do containers give enough compute isolation for tenants running their own code? TRICKY

Usually not: containers share the host kernel, so untrusted code calls for sandboxed runtimes or micro-VMs (gVisor, Kata, Firecracker) or dedicated nodes.

Next: SD-R-002 · Shared vs Dedicated Databases backendroadmap.com